

HOMEScout AI — PUNE HOUSE PRICE PREDICTOR

An end-to-end machine learning case study — from raw Pune property listings to a deployed prediction app with a self-updating, cloud-backed insights dashboard

Author	Aditya Devkar
Tool(s)	Python, Streamlit, Scikit-learn, Supabase (PostgreSQL), SQLAlchemy, Plotly Express
Project Type	Regression modelling + full-stack data application (multi-page web app)
Domain / Market	Residential real estate — Pune, Maharashtra, India
Deployment	Streamlit Community Cloud (live, publicly accessible)
Repository	github.com/adityadevkar/HomeScout_AI
Live Application	homescoutai.streamlit.app

1. Project Overview

Buying a home in Pune is an information problem before it is a financial one. Asking prices vary widely between localities, listing portals quote different numbers for comparable properties, and a first-time buyer has no neutral reference point for what a given configuration should actually cost. HomeScout AI closes that gap with a single, interactive web application: the user enters four details about the property they have in mind — locality, house type, built-up area and number of bedrooms — and immediately receives a data-driven price estimate in lakhs, alongside one-click links to live listings matching those exact criteria on India's major property portals.

The project goes a step beyond a standard prediction demo. Every search a user runs is written to a cloud-hosted PostgreSQL database, and a second page of the application reads that same database back to produce a live analytics dashboard of user search behaviour. The result is a closed loop: the application does not just serve predictions, it generates its own proprietary demand-side dataset and visualises it in real time — which is exactly the kind of feedback loop a real property-tech product would be built on.

2. The Business Question

The project was framed around two distinct audiences and the questions each of them brings to the property market:

For the home buyer (the prediction layer):

- What is a fair price for a property of this size, type and configuration in this specific Pune locality?
- How much does location alone change the price, holding area and BHK constant?
- Where can I see real listings that match these criteria right now, without re-entering my filters on five different websites?

For the business / product owner (the analytics layer):

- Which Pune localities are buyers searching for most?
- What configuration is in demand — is the market looking for 1, 2, 3 or 4+ BHK homes?
- What price bracket are users actually shopping in, and how does the average predicted price differ by locality?
- Is search volume growing, and which segments should inventory and marketing be aimed at?

3. Data

3.1 Source — Property Listings

The predictive model is trained on a Pune residential property listings dataset covering sale listings across the city's localities. Each record describes one property and carries the attributes needed to explain its asking price:

- **Location** — the Pune locality or suburb in which the property is listed (the dominant price driver in any Indian metro).
- **House Type** — one of five property formats: Apartment, Independent Floor, Independent House, Studio Apartment or Villa.
- **Area** — built-up area of the unit in square feet.
- **BHK** — number of bedrooms (bedroom-hall-kitchen configuration), the standard Indian sizing convention.
- **Price** — the listed sale price, which is the target variable the model learns to predict. Prices are reported to the user in lakhs (₹100,000 units), the convention buyers in India actually use.

All exploration, cleaning, feature engineering and model training for this layer are contained in `Project.ipynb` in the repository.

3.2 Model Feature Schema

The trained model does not consume raw text. Categorical fields are one-hot encoded during preparation, and the resulting column order is frozen and persisted to `columns.json` so that the application can rebuild an identical feature vector at prediction time. The served schema is:

Feature Block	Description
Index 0 — area	Numeric. Built-up area in square feet.
Index 1 — bhk	Numeric. Number of bedrooms.
House type flags	Five binary columns: apartment, independent floor, independent house, studio apartment, villa. Exactly one is set to 1.
Location flags	One binary column per Pune locality in the training data. Exactly one is set to 1.

At runtime the application builds a zero vector the length of the full column list, writes area and BHK into the first two positions, and flips the single location flag and single house-type flag that correspond to the user's

selection before calling the model. This design keeps training and inference perfectly aligned and removes any need to ship an encoder object alongside the model.

3.3 Application Data Model — the Searches Table

The second dataset in this project is generated by the application itself. On first run, the app creates (if it does not already exist) a searches table in the Supabase PostgreSQL instance, and appends one row per prediction:

Column	Description
id	Serial primary key.
timestamp	Date and time the search was executed — enables trend analysis over time.
location	Pune locality the user selected.
house_type	Property format selected.
area_sqft	Area entered by the user, in square feet.
bhk	Bedroom configuration selected.
predicted_price_lakhs	The model's output for that search, stored in lakhs.

This table is the single source of truth for the Insights dashboard. Because it is written on every prediction and read live on every dashboard load, the analytics are never stale and require no manual refresh, export or scheduled job.

4. Data Cleaning & Preparation

Raw property listing data is notoriously untidy — inconsistent locality spellings, mixed area units, extreme outliers from luxury and mis-entered listings, and a long tail of localities with only one or two records. The preparation pipeline addressed these before any model was fit:

- **Standardising data types and text fields** — trimming whitespace and normalising all location and house-type labels to a single lower-case convention, so that the same locality is never treated as two different categories.
- **Handling missing and invalid records** — removing or imputing rows with missing area, BHK or price, since none of these can be inferred safely for a price model.
- **Normalising area and price to consistent units** — reducing all area values to square feet and all prices to a single rupee base, then converting to lakhs only at the presentation layer.
- **Outlier treatment** — filtering listings with implausible price-per-square-foot values or unrealistic area-to-BHK ratios, which would otherwise dominate a squared-error loss and distort estimates for ordinary homes.
- **Consolidating the long tail of localities** — grouping very low-frequency locations so the model is not fitting a dedicated coefficient to a locality seen only once or twice.

- **One-hot encoding and column freezing** — converting location and house type into binary indicator columns, then exporting the exact final column order to columns.json so training and production inference cannot drift apart.
- **Input guard-rails in the app** — the user interface constrains area to 300–5,000 sq. ft. and BHK to 1–5, and offers location and house type only as dropdowns built from the model's own column list. This makes it structurally impossible for a user to submit a feature combination the model was never trained to handle.

5. Modelling & Methodology

With a clean feature matrix in place, the problem was framed as supervised regression: predict a continuous sale price from four property attributes.

- **Algorithm** — a Scikit-learn regression model, selected by comparing candidate regressors on held-out data within the project notebook.
- **Target** — listed sale price, surfaced to the end user in lakhs for readability.
- **Evaluation** — the model was scored on unseen test data before being accepted, with residual behaviour inspected across price bands to confirm it was not systematically over- or under-estimating at either end of the market.
- **Serialisation** — the fitted model is persisted with pickle to the file `pune_house_price_model` and loaded once at application start-up, so no training work happens at request time and predictions return effectively instantly.
- **Inference contract** — the `predict_price()` helper in the application rebuilds the frozen feature vector from the user's four inputs and returns the model's estimate, failing safely if a selection cannot be mapped to a known column.

The methodology deliberately favours a transparent, reproducible model with a small, business-meaningful feature set over a heavier black-box approach — the four inputs chosen are precisely the four a buyer can state with confidence before ever visiting a property.

6. The Application

The deliverable is a two-page Streamlit application with a custom purple-gradient theme, glassmorphic input panels and Poppins typography — styled well beyond Streamlit defaults through an embedded CSS layer.

6.1 Page 1 — Home (Prediction & Live Listings)

The primary workflow. A bordered two-column form collects Location, Area (sq. ft.), House Type and Bedrooms (BHK). A single action button runs the prediction, displays the estimate in a highlighted metric card formatted as ₹ X.XX Lakhs, and then renders a set of styled listing bars linking straight out to matching searches on live property portals.

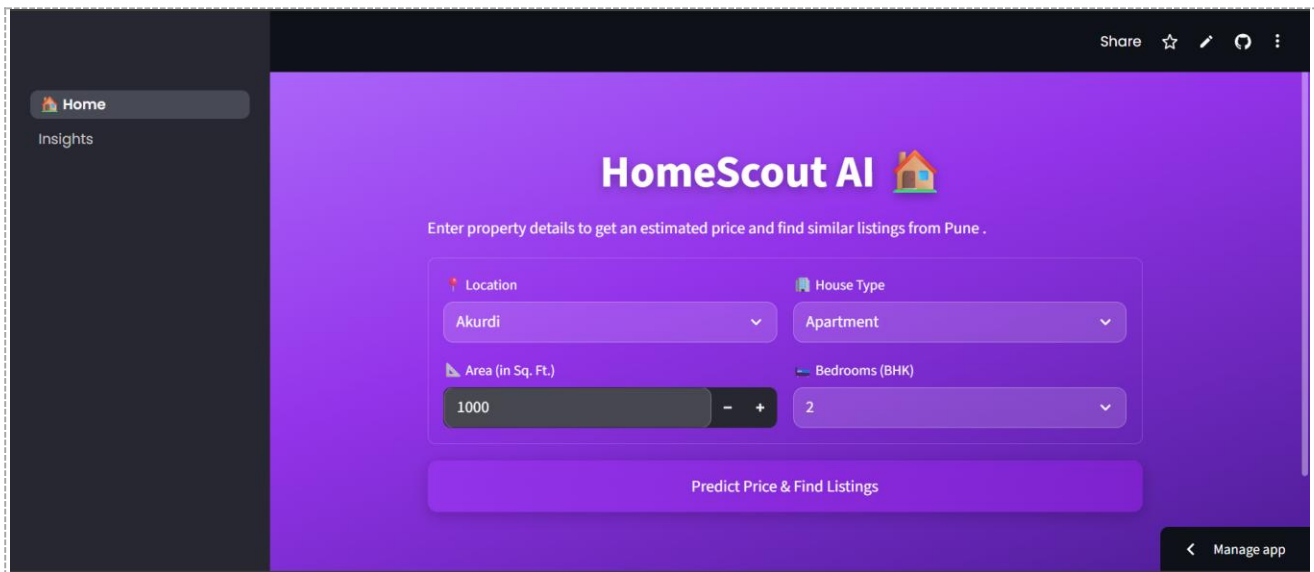


Fig. 1 — Home page: property input form, predicted price, and live listing shortcuts.

6.2 Page 2 — Insights (Live Analytics Dashboard)

A separate page that queries the Supabase database for the full history of user searches and renders it through Plotly Express. It reports:

- Top-level metrics — total searches recorded and the most popular searched location.
- Top searched locations — ranking Pune localities by user demand.
- BHK distribution — a pie chart of the configurations buyers are looking for.
- Average predicted price by location — a comparative view of what the model estimates across the city's localities.

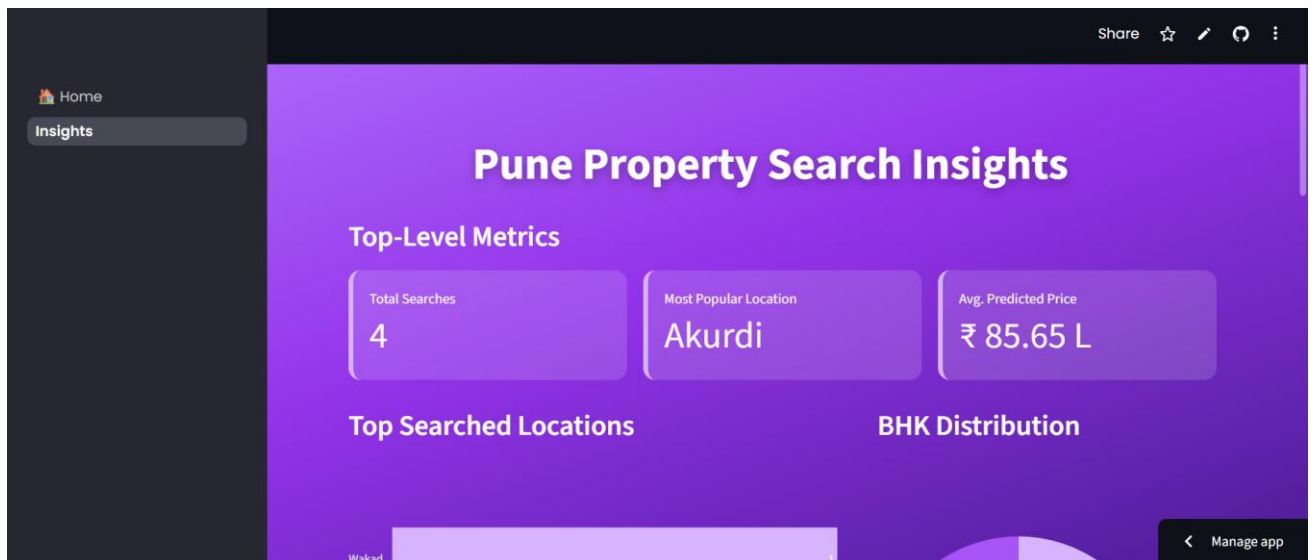


Fig. 2 — Insights page: live dashboard of historical user searches from the Supabase database.

6.3 Live Listing Integration

Rather than scraping portals — which is fragile, rate-limited and legally awkward — the application constructs deep search URLs on the fly by slugifying the selected locality and injecting the BHK filter. From a single prediction the user gets pre-filtered searches on five portals:

- 99acres, MagicBricks, Housing.com, NoBroker and PropertyWala.

Each opens in a new tab with the user's locality and bedroom count already applied, turning an abstract price estimate into an immediately actionable shortlist.

7. System Architecture & Data Flow

The project is built on a lightweight, fully cloud-native stack with no server to manage:

1. User input — the buyer enters locality, area, house type and BHK on the Streamlit Home page.
2. Feature construction — the app rebuilds the frozen feature vector from columns.json.
3. Prediction — the pickled Scikit-learn model held in memory returns an estimated price.
4. Presentation — the estimate is displayed in lakhs and dynamic portal search links are generated.
5. Database write — the full search (inputs plus prediction) is appended to the searches table in Supabase PostgreSQL via SQLAlchemy and psycopg2, over an SSL-required connection.
6. Database read — when the user opens the Insights page, the app queries the same table for all historical searches.
7. Visualisation — Plotly Express renders the metrics and charts from that live result set.

Credentials are never committed. The connection is assembled at runtime from Streamlit's secrets store (`st.secrets["db_credentials"]`), and the app connects through Supabase's transaction pooler, which is the correct choice for a stateless, many-short-connections application like this one.

8. Key Outcomes

- **A working end-to-end ML product, not a notebook.** The project spans data cleaning, model training, serialisation, UI, persistence and analytics — and is deployed publicly, so anyone can use it without installing anything.
- **Location is the dominant price signal.** Because every Pune locality is given its own model coefficient, two identical-sized homes can return materially different estimates purely on address — mirroring how the real market prices property.
- **The application creates its own data asset.** Each prediction adds a timestamped, structured record of genuine buyer intent. Over time this search log becomes more commercially interesting than the listings data it was built on, because it captures demand rather than supply.
- **Analytics are live by construction.** The dashboard reads the production database directly, so there is no refresh schedule, no extract, and no possibility of the numbers being out of date.
- **Prediction is connected to action.** By pairing the estimate with pre-filtered portal searches, the app closes the loop between "what should this cost?" and "show me what is available" in a single click.
- **Training and inference cannot drift.** Freezing the encoded column order to columns.json and rebuilding the vector from it at runtime eliminates the single most common cause of silent prediction errors in deployed models.

9. Deployment & How to Run Locally

The application is deployed on Streamlit Community Cloud and is live at homescoutai.streamlit.app. To run it locally:

8. Clone the repository and change into the project directory.
9. Create and activate a virtual environment (recommended).
10. Install dependencies with `pip install -r requirements.txt`.
11. Create a free Supabase project, open Database → Connection → Transaction pooler, and copy the pooler credentials.
12. Create a `.streamlit/secrets.toml` file in the project root containing a `[db_credentials]` block with user, password, host, port and dbname.
13. Launch the app with `streamlit run 0_🏠_Home.py`.

The searches table is created automatically on first run, so no manual schema setup is required.

10. Limitations & Future Enhancements

Known limitations:

- The model is trained on listed asking prices, not recorded transaction prices, so estimates reflect what sellers ask rather than what buyers finally pay.
- Only four features are used. Floor level, property age, furnishing status, amenities and proximity to transport all move real prices and are not currently modelled.
- Coverage is limited to localities present in the training data; genuinely new or very rare micro-markets cannot be predicted.
- Because the model is static, estimates will drift from the market until the model is retrained on refreshed listings.

Planned enhancements:

- Surface a confidence interval or price range alongside the point estimate, so users see the uncertainty rather than a single deceptively precise number.
- Enrich the feature set with property age, floor, furnishing and amenity flags.
- Add a scheduled retraining pipeline so the model tracks the market rather than a snapshot of it.
- Extend the Insights dashboard with a time-series of search volume and a searched-price-band distribution.
- Generalise the application beyond Pune to other Indian metros using the same architecture.

11. Tools & Tech Stack

Category	Tools Used
Language	Python
Web Framework / UI	Streamlit (multi-page app, custom CSS theming, Poppins web font)

Category	Tools Used
Machine Learning	Scikit-learn (regression model), pickle (model serialisation)
Data Manipulation	Pandas, NumPy
Database	Supabase — hosted PostgreSQL (transaction pooler, SSL required)
Database Connectivity	SQLAlchemy, psycopg2-binary
Visualisation	Plotly Express
Configuration & Secrets	Streamlit secrets (secrets.toml), columns.json feature schema
Deployment	Streamlit Community Cloud
Development & Version Control	Jupyter Notebook (Project.ipynb), Git and GitHub, Dev Containers

12. Repository Structure

Path	Purpose
0_🏠_Home.py	Main Streamlit entry point — UI, styling, model loading, prediction, listing URL generation and database writes.
pages/	The Insights dashboard page, which reads the Supabase database and renders the Plotly analytics.
Project.ipynb	Exploratory analysis, data cleaning, feature engineering and model training notebook.
pune_house_price_model	The serialised (pickled) trained regression model.
columns.json	The frozen encoded feature column order used to rebuild the input vector at inference.
requirements.txt	Python dependencies: streamlit, pandas, numpy, scikit-learn, sqlalchemy, psycopg2-binary, plotly.
Homepage.png / Insightspage.png	Application screenshots used in documentation.
.devcontainer/	Dev container configuration for a reproducible development environment.

13. Links

GitHub Repository: https://github.com/adityadevkar/HomeScout_AI

Live Application: <https://homescoutai.streamlit.app/>